ACTIVIDAD

Dosier de Actividades del Tema: *Testing*

DAM / DAW

Módulo 5 Entornos de desarrollo

Linkia FP

EJERCICIO 1

Complejidad ciclomática

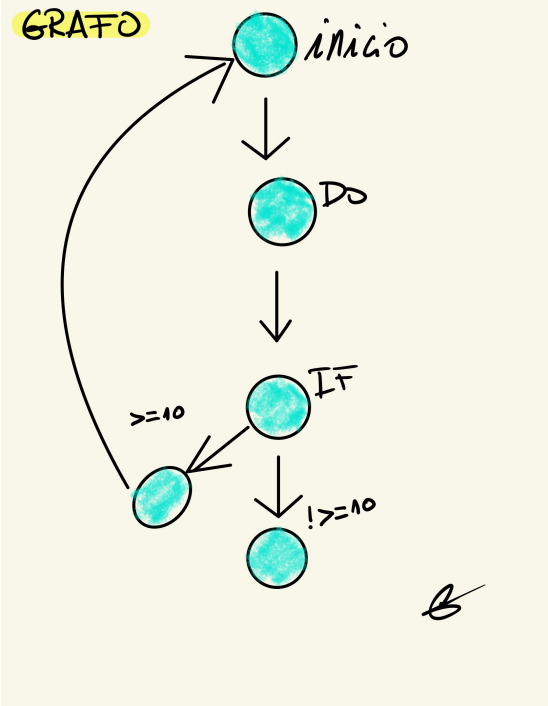
En esta actividad, calcularemos la complejidad ciclomática de un programa.

1. Crea el grafo de flujo del código adjunto abajo (adjunta evidencia).
2. Calcula la complejidad ciclomática del código.
3. Identificar los caminos independientes del código.

Bloque de código a utilizar en la actividad:

```
        Scanner sc = new Scanner(System.in);
        System.out.println("Escribe una palabra");
        String palabra = sc.nextLine();
        System.out.println("Tu palabra es: " + palabra);
        boolean error = false;
        do {
            sc.nextLine();
            error = false;

                System.out.println("Escribe un número entero menor de 10:");
                int num = sc.nextInt();

             if(num>=10) {
                error = true;
                 }

       } while (error);

     System.out.println("Tu número: " + num);
}
```


Linkia FP



Grafo de flujo del código:

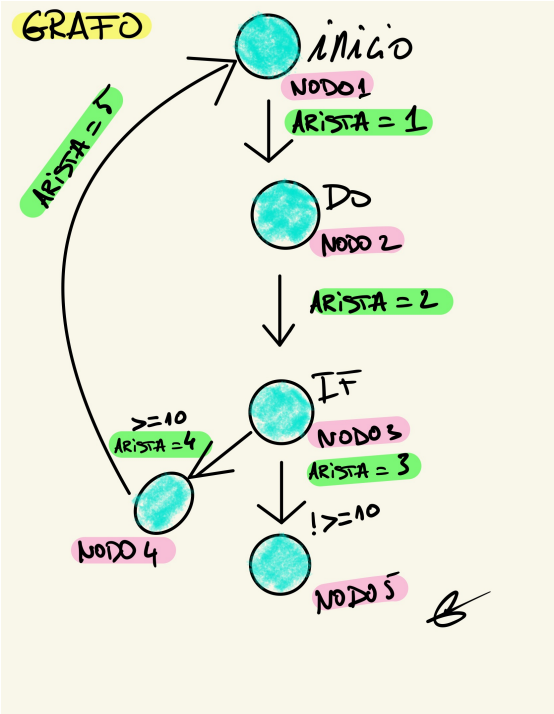
Nodo de inicio.

Nodo Do —> inicio del bucle

Nodo if —> condición >= 10

Nodo de salida !>=10 —> indica cuando se cumple la condición de salida.

El bucle regresa al principio si num >=10



Calcular la complejidad ciclomática:

Usamos la fórmula:

V(G) = E - N + 2

E es el número de aristas o flechas

N es el número de nodos

Resultado:
5 (E) - 5 (N) + 2 = 2

Disponemos de dos caminos independientes de código, lo que significa que necesitaríamos al menos 2 pruebas unitarias para recorrer todos los caminos posibles en una prueba de caja blanca.

A continuación identifico y documento los caminos:

Camino 1: El programa pide una palabra, entra en el bucle do-while. Pide un número. Este es menor que 10, sale de bucle. Imprime el número. Fin.

Camino 2: El programa pide una palabra, entra en el bucle do-while. Pide un número. Este es mayor o igual que diez. Se repite el bucle.

EJERCICIO 2

Dado el código Java de la clase Punto2D (que te incluyo a continuación), implementar las pruebas unitarias que permiten comprobar el correcto funcionamiento de la clase Punto2D. Para esto, usaremos Eclipse y la librería JUnit.

1. Crear una clase de prueba (New JUnit Test Case) llamada TestPunto2D (un caso de prueba JUnit) sobre la clase Punto usando el código adjunto abajo.

2. Implementar los diferentes métodos de la clase TestPunto2D.

    a. Añade al menos un comentario en cada método explicando brevemente que hace la prueba.

    b. Adjunta evidencia del código

3. Ejecutar el test

    a. comprobar que no hay errores en tus pruebas.

    b. Adjuntar una captura de pantalla de la ejecución de los tests implementados en jUnit.

Adjuntar el código de la clase TestPunto2D con los tests implementados.

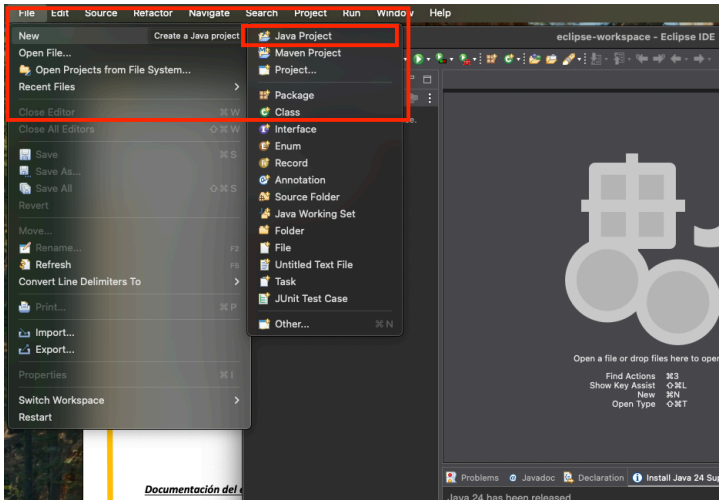
Bloque de código a utilizar en la actividad:

```
public class Punto2D {

	private double x;
	private double y;

	public Punto2D() {
		x=0;
		y=0;
	}
	public void moverX(double l)
	{
		x=x+l;
	}
	public void moverY(double l)
	{
		y=y+l;
	}
	public double producto_escalar() {
		return (x*y);
	}
	public void print() {
```

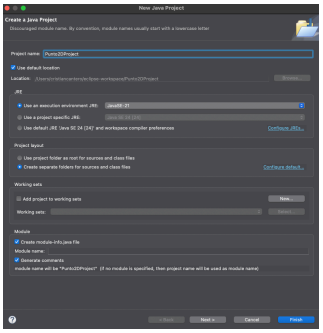

Linkia FP

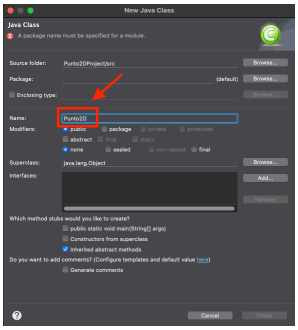
```
            System.out.print("("+ x + "," + y +")" );

            }
      public double getx() {
            return x;
      }
      public double gety() {
            return y;
      }
      public void setx(double a) {
            x=a;
      }
      public void sety(double b) {
            y=b;
      }

}
```

Documentación del ejercicio.



Abrimos Eclipse y vamos a File —> new —> Java Project





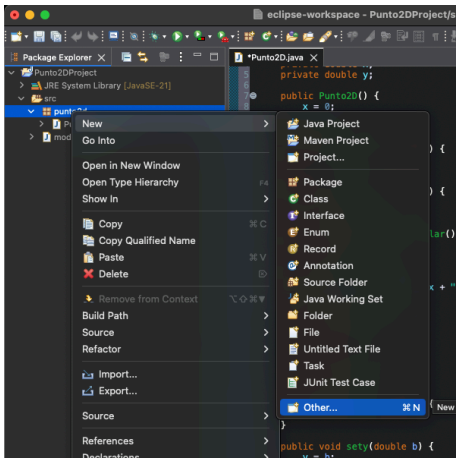
Creamos el proyecto donde haremos todo el ejercicio. Y lo nombraremos.

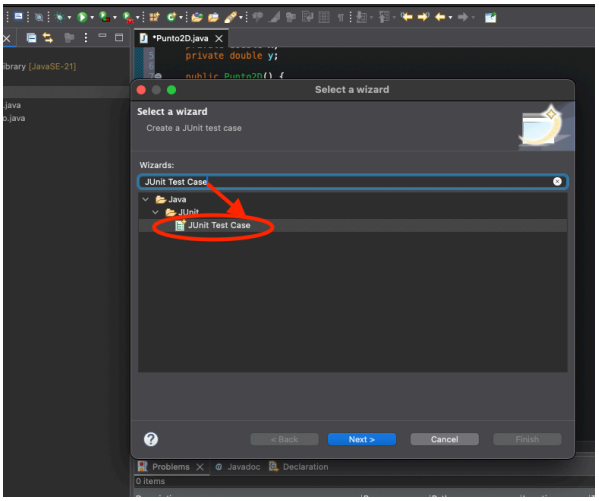
Crear la clase de test TestPunto2D usando JUnit

Una vez copiado el código del ejercicio en el IDE procedemos a desarrollar este apartado.

Como crear un JUnit Test Case.

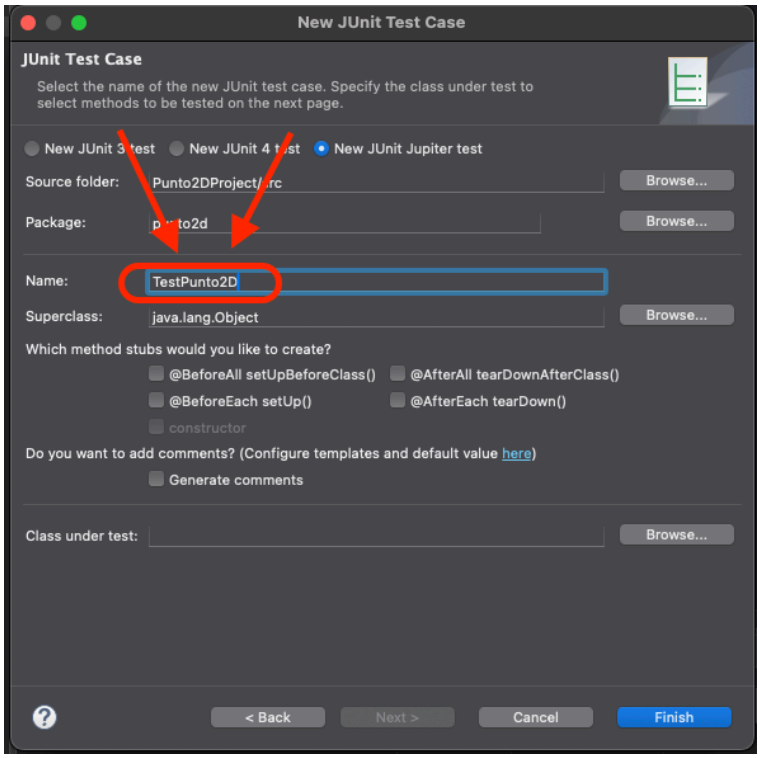
Click derecho sobre el paquete punto2d.
Seleccionamos New>Other.... ==========>



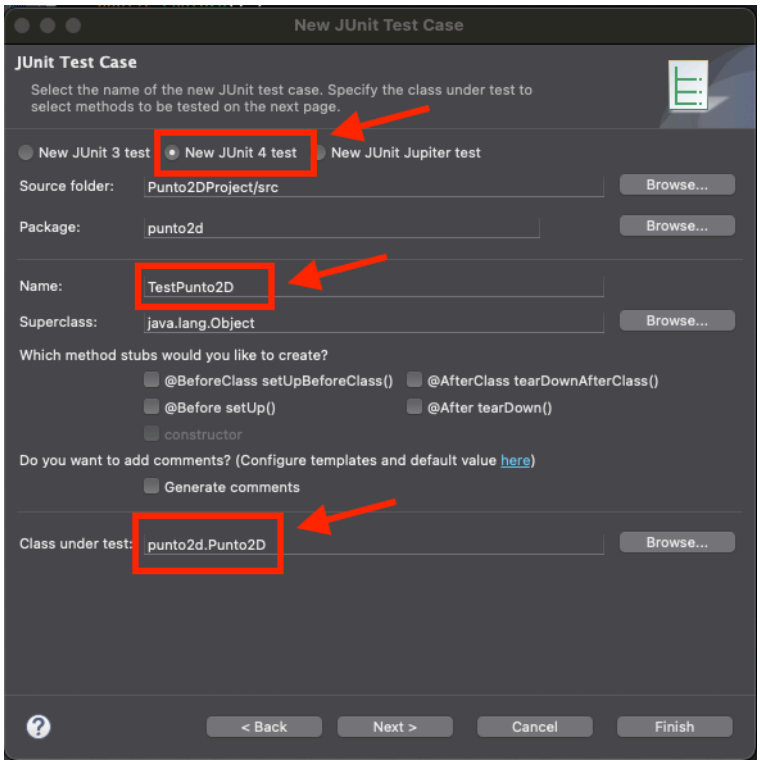


En el buscador pondremos JUnit Test Case...

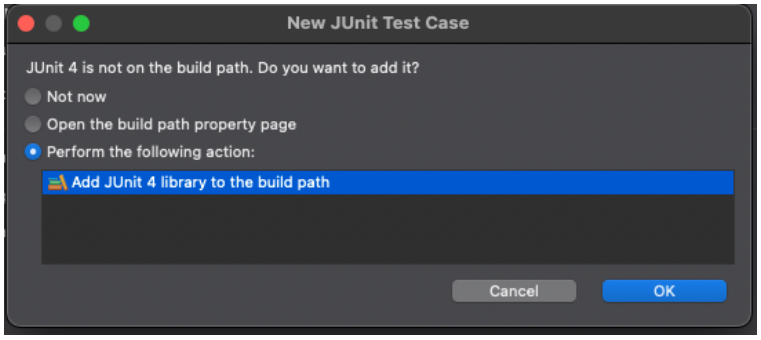
Y le damos a Next...



Ya podemos ponerle nombre al Test…



Esto es importante. Tanto el New JUnit 4 Test marcado, como el nombre y especificarle que es una clase “under test”. Hacemos browse y lo agregamos manualmente.

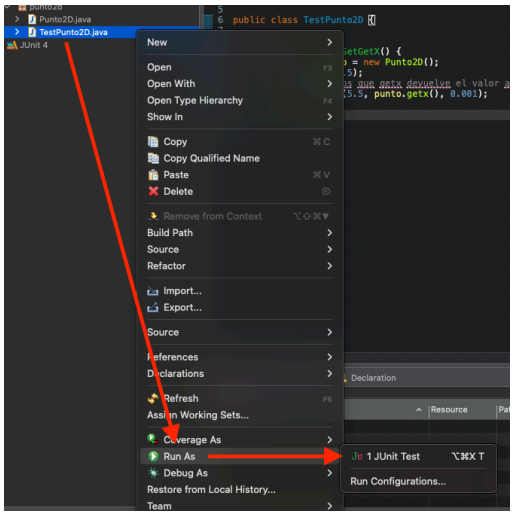


Le decimos que queremos agregar la librería y OK.

NOTA: Tenía un problema que no paraba de darme warnings y tuve que cargarme el archivo module-info.class

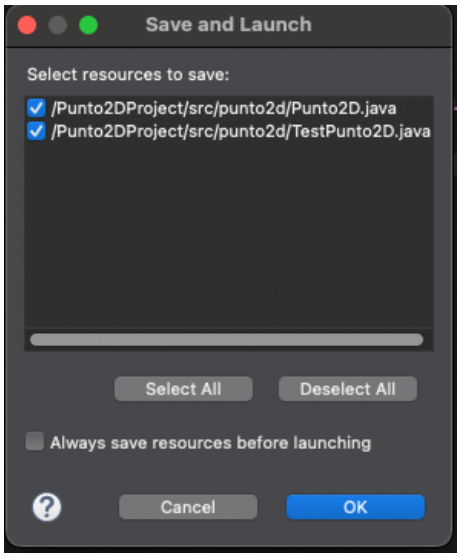
Como Eclipse está configurado con Java modular, y JUnit4 no lo es, lo más práctico y funcional para el ejercicio fue quitar el módulo.(Fuente: diosgpt)

EJECUTAR EL TEST testSetGetX

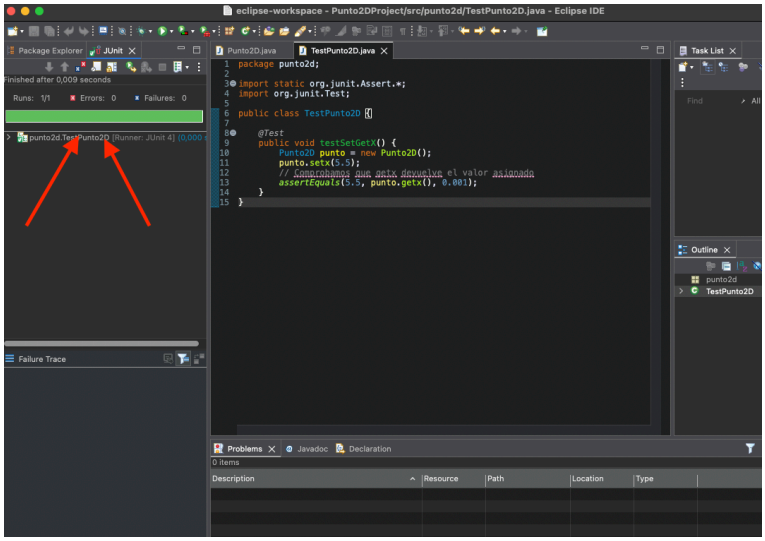


Click derecho en el archivo TestPunto2D.java

Run As > Unit Test



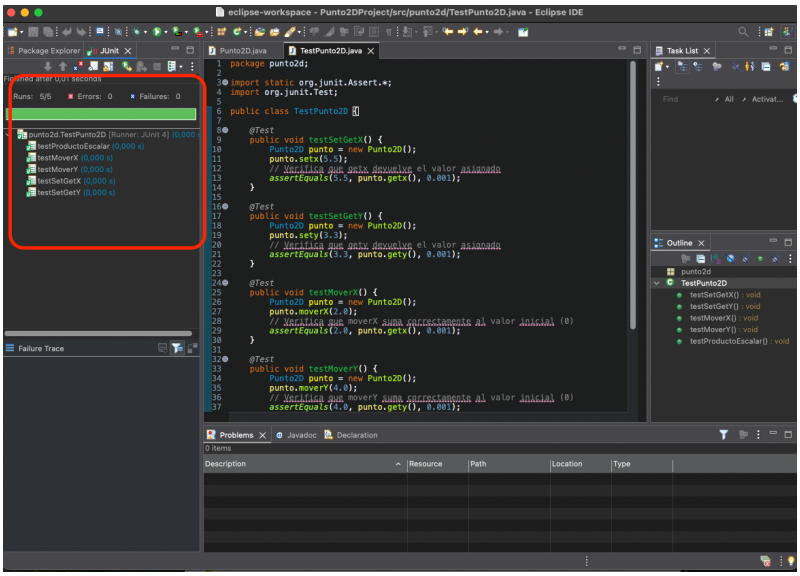
Cruzamos los dedos... 🤞



En esta vista de JUnit vemos la barra verde que nos indica que el procedimiento ha sido el correcto.

Ahora que vemos que funciona ya podemos hacer el resto de pruebas.

Todo correcto así que ya tenemos hechos los test!!!



Linkia FP